REPORT

Reviewer report: andialbrecht/sqlparse#936: Fix

statement splitting after whitespace-varied compound

closers

Run sqlparse-936, tested 2026-11-09 11:45 IST by ops/daily_run.py (route source-pair). Proof-of-Change

commit ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 12:03 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: High

The PR's tests behave the same or better than on main, in all 3 runs per side. Every check passed and every

claim cites a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Fix statement splitting after whitespace-varied compound closers

(andialbrecht/sqlparse#936).

• What we observed: 30 test(s) fail on main and pass on the PR; 79 test(s) ran on main, 89 on the PR; 10 new

test(s) pass, 49 unchanged; probe: 1 unchanged sub-check(s) of an input where the claim's change was seen,

shown as advisory only: c2.original_token_text_unchanged (C2); probe: 6 observation(s) shown as advisory only

(claim not shown on these inputs; a related change was proven): c1.statement_count_in_regression_case (C1);

c3.block_types (C3); c3.closer_types (C3); c3.following_statement_counts (C3) (+2 more); probe: claim(s) C2,

C4 proven.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 6.5 GB (of 15.6 GB).

• Setup check: free disk 92.9 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: sqlparse python setup install instructions https://github.com/andialbrecht/sqlparse site:github.com/andialbrecht/sqlparse

• Query: sqlparse python documentation install https://github.com/andialbrecht/sqlparse site:github.com/andialbrecht/sqlparse

• Query: sqlparse python install requirements https://pypi.org/project/sqlparse/ https://github.com/andialbrecht/sqlparse site:pypi.org

• Source: sqlparse · PyPI — https://pypi.org/project/sqlparse/

• Source: GitHub - andialbrecht/sqlparse: A non-validating SQL parser module for Python · GitHub — https://github.com/andialbrecht/sqlparse

• Source: sqlparse/Makefile at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/Makefile

• Source: sqlparse/.github/workflows/python-app.yml at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/.github/workflows/python-app.yml

• Source: sqlparse/CHANGELOG at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/CHANGELOG

• Source: sqlparse/.readthedocs.yaml at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/.readthedocs.yaml

• Setup plan: the same packages on main and pr.

• Setup plan: install build (listed in the dev extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10

• Installed build 1.6.1 from pyproject (listed in the dev extra) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (49 unchanged, 3 runs per side) | pytest-pr-1.log:8 «89 passed in 0.59s»

| P-C1 | Statement splitting works correctly after whitespace■varied compound closers (the fix resolves the

merging of later statements). (PR description) | not observed | probe model-probe1,

c1.statement_count_in_regression_case: same on both sides: 1 | probe-model-probe1-pr-1.jsonl:1

«"observation": "c1.statement_count_in_regression_case"»

| P-C2 | Only the keyword value used for splitter comparisons is normalized; the original token text remains

unchanged. (PR description) | not observed | probe model-probe1, c2.original_token_text_unchanged: same on

both sides: "end if" | probe-model-probe1-pr-1.jsonl:3 «"observation": "c2.original_token_text_unchanged"»

| P-C3 | Regression coverage was added for five closer types, four separators, anonymous and procedure

blocks, and two following statements. (PR description) | not observed | probe model-probe1, c3.block_types:

same on both sides: 2 | probe-model-probe1-pr-1.jsonl:4 «"observation": "c3.block_types"»

| P-C4 | On pristine master, the new regression cases fail 30 out of 40, while the 10 single■space controls pass.

(PR description) | proven | probe model-probe1, c4.control_case_passed: same on both sides: 0 |

probe-model-probe1-pr-1.jsonl:8 «"observation": "c4.control_case_passed"»

| P-C5 | With the change applied, all 40 regression cases pass. (PR description) | not observed | probe

model-probe1, c5.regression_case_passed: same on both sides: 0 | probe-model-probe1-pr-1.jsonl:10

«"observation": "c5.regression_case_passed"»

Citation check (PASS): 6 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 79 / 89 | -

| Tests unchanged | 49 | | New tests that pass | 10 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 30 |

tests.test_split::test_split_compound_closing_keyword_whitespace[CASE 1 WHEN 1 THEN-CASE--CREATE

PROCEDURE p()], tests.test_split::test_split_compound_closing_keyword_whitespace[CASE 1 WHEN 1

THEN-CASE- -], tests.test_split::test_split_compound_closing_keyword_whitespace[CASE 1 WHEN 1

THEN-CASE-\n-CREATE PROCEDURE p()],

tests.test_split::test_split_compound_closing_keyword_whitespace[CASE 1 WHEN 1 THEN-CASE-\n-],

tests.test_split::test_split_compound_closing_keyword_whitespace[CASE 1 WHEN 1 THEN-CASE-\t-CREATE

PROCEDURE p()]

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe1.py, frozen

9d5ed28f9f17 at 2026-11-09T06:17:30.310005+00:00): Tests statement splitting after whitespace-varied

compound closers and related claims.

• P-C1 (PR description): Statement splitting works correctly after whitespace■varied compound closers (the fix

resolves the merging of later statements). Expects the covered observations to differ.

• P-C2 (PR description): Only the keyword value used for splitter comparisons is normalized; the original token

text remains unchanged. Expects the covered observations to differ.

• P-C3 (PR description): Regression coverage was added for five closer types, four separators, anonymous and

procedure blocks, and two following statements. Expects the covered observations to differ.

• P-C4 (PR description): On pristine master, the new regression cases fail 30 out of 40, while the 10

single■space controls pass. Expects the covered observations to stay the same.

• P-C5 (PR description): With the change applied, all 40 regression cases pass. Expects the covered

observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.statement_count_in_regression_case | P-C1 | no change although the claim says it changes | same on both

sides: 1 | probe-model-probe1-main-1.jsonl:1 «"observation":

"c1.statement_count_in_regression_case"»
probe-model-probe1-pr-1.jsonl:1 «"observation":

"c1.statement_count_in_regression_case"»

| c2.block_stack_change_at_closer | P-C2 | as claimed | value: 0 -> -1 | probe-model-probe1-main-1.jsonl:2

«"observation": "c2.block_stack_change_at_closer"»
probe-model-probe1-pr-1.jsonl:2 «"observation":

"c2.block_stack_change_at_closer"»

| c2.original_token_text_unchanged | P-C2 | no change although the claim says it changes | same on both sides:

"end if" | probe-model-probe1-main-1.jsonl:3 «"observation":

"c2.original_token_text_unchanged"»
probe-model-probe1-pr-1.jsonl:3 «"observation":

"c2.original_token_text_unchanged"»

| c3.block_types | P-C3 | no change although the claim says it changes | same on both sides: 2 |

probe-model-probe1-main-1.jsonl:4 «"observation": "c3.block_types"»
probe-model-probe1-pr-1.jsonl:4

«"observation": "c3.block_types"»

| c3.closer_types | P-C3 | no change although the claim says it changes | same on both sides: 5 |

probe-model-probe1-main-1.jsonl:5 «"observation": "c3.closer_types"»
probe-model-probe1-pr-1.jsonl:5

«"observation": "c3.closer_types"»

| c3.following_statement_counts | P-C3 | no change although the claim says it changes | same on both sides: 2 |

probe-model-probe1-main-1.jsonl:6 «"observation":

"c3.following_statement_counts"»
probe-model-probe1-pr-1.jsonl:6 «"observation":

"c3.following_statement_counts"»

| c3.separators | P-C3 | no change although the claim says it changes | same on both sides: 4 |

probe-model-probe1-main-1.jsonl:7 «"observation": "c3.separators"»
probe-model-probe1-pr-1.jsonl:7

«"observation": "c3.separators"»

| c4.control_case_passed | P-C4 | as claimed | same on both sides: 0 | probe-model-probe1-main-1.jsonl:8

«"observation": "c4.control_case_passed"»
probe-model-probe1-pr-1.jsonl:8 «"observation":

"c4.control_case_passed"»

| c4.regression_case_passed | P-C4 | as claimed | same on both sides: 0 | probe-model-probe1-main-1.jsonl:9

«"observation": "c4.regression_case_passed"»
probe-model-probe1-pr-1.jsonl:9 «"observation":

"c4.regression_case_passed"»

| c5.regression_case_passed | P-C5 | no change although the claim says it changes | same on both sides: 0 |

probe-model-probe1-main-1.jsonl:10 «"observation":

"c5.regression_case_passed"»
probe-model-probe1-pr-1.jsonl:10 «"observation":

"c5.regression_case_passed"»

Citation check (PASS): 20 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (1.6 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| sqlparse/engine/statement_splitter.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 8ec1af1f4665 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 6 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 20 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

Nothing went wrong.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 6 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 20 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-08/sqlparse-936 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/andialbrecht/sqlparse/pull/936

Evidence links

• Pull request andialbrecht/sqlparse#936

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe1-main-1.jsonl

• probe-model-probe1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:7 «49 passed in 0.59s»

• pr run 1 result: pytest-pr-1.log:8 «89 passed in 0.59s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 12:04 IST